

Vedant Vartak

DATA 260 - Agentic AI and Distributed Systems

Homework 5: Tools, Reliability, and a Safe Local Agent

GitHub: <https://github.com/vedantvartakwork/data260-3539>

Implementation commit: f4990b6c1c2cdee70ea7379ec83269b712449372

Submission tag: hw5 (applied to the final report commit)

Item	Value	Item	Value
SID4	3539	PORT_BASE	8839
PREFIX	s3539	SEED	3539
VERIFY_SEED	263539	DOMAIN_ID	3
Domain	Grocery supply and recall notices	Local model	qwen3:8b
Hardware	Apple M4 MacBook Air	Memory / CPU	16 GB / 10 cores
Platform	macOS 15.3	Ports	API 8839 / UI 5173

This cumulative submission extends the authenticated FastAPI/MySQL application with a manufacturer relationship, Redux Toolkit and Axios, two MCP servers, strict tool schemas, bounded retries, and a local Ollama tool-calling loop with deterministic safety enforcement.

Submission contents

The repository includes RUN_LOG.txt, METRICS.md, REFLECTION.md, TOOL_CONTRACTS.md, AI_USE.md, verification.json, 150 raw fault-injection rows, MCP/agent/API raw outputs, Postman collection and environment files, and this screenshot-backed report.

1. Completion and verification

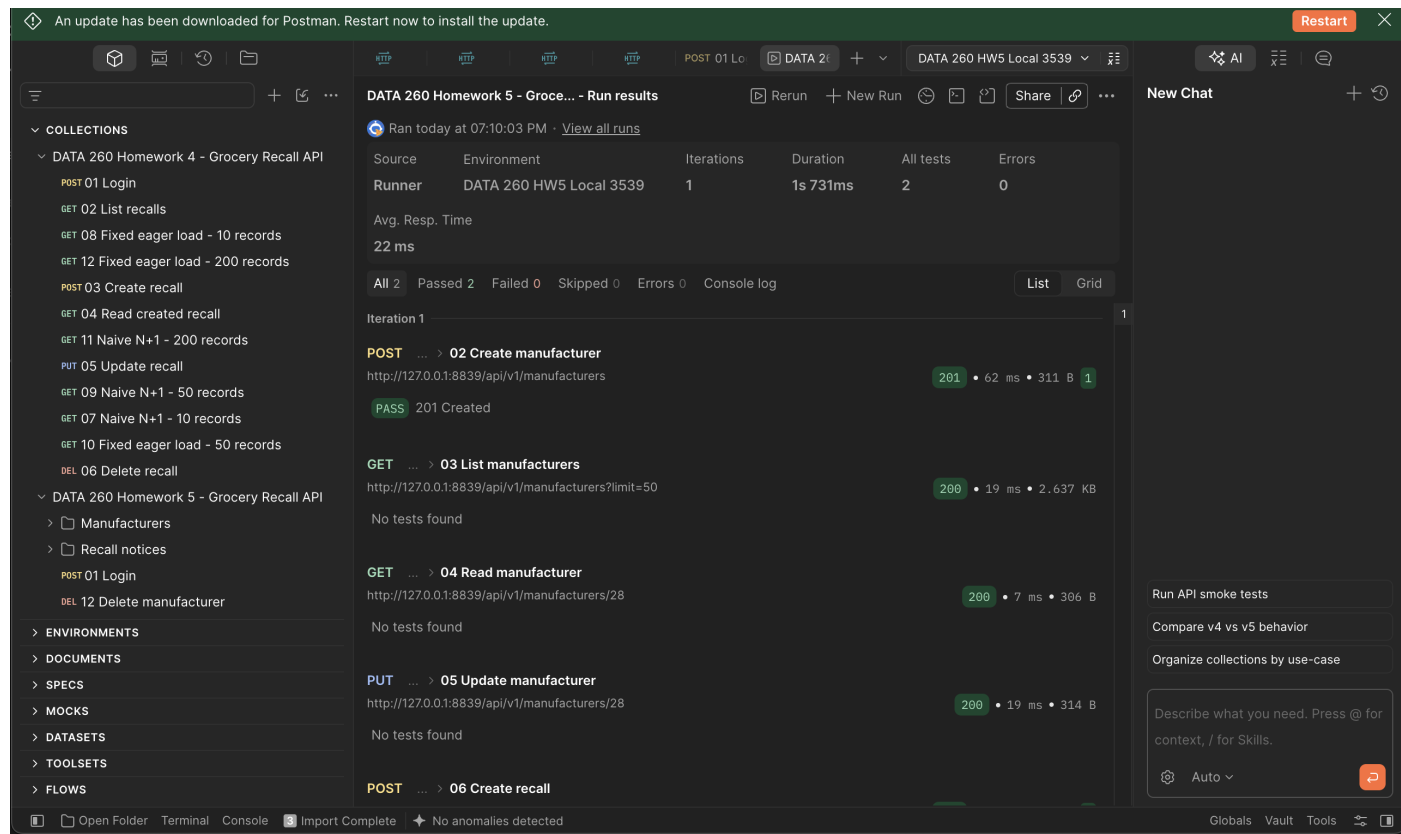
The final cumulative checks passed without exceptions or skipped requirements.

Area	Evidence	Result
Database/API	12 manufacturers; 500 recalls; 200 events; authenticated CRUD	PASS
React/Redux	Home, create, update, delete; Redux Toolkit thunks; Axios	PASS
MealDB MCP	Exactly four live tools demonstrated	PASS
Domain MCP	Exactly search/detail/aggregate; valid and invalid calls	PASS
Reliability	150 deterministic calls; 3-attempt bounded retry	PASS
Offline tests	9/9 tool and safety tests	PASS
Cumulative suite	33/33 Python tests; React production build	PASS
Local agent	Four qwen3:8b scenarios; one enforced safety block	PASS
Verifier	12/12 checks	PASS

The verifier independently inventories both MCP servers, calls one tool from each, checks the live API health endpoint, counts all 150 experiment rows, runs the offline runner and 33-test suite, and rebuilds the React client.

2. FastAPI and Postman CRUD - first half

A single collection run used the SID-specific environment and completed with zero errors.



Postman run summary and successful manufacturer create/list/read/update plus recall create/list/read operations.

3. FastAPI and Postman CRUD - second half

The same run verifies the relationship endpoint, update path, authenticated login, and cleanup deletes.

The screenshot shows the Postman interface with a collection named 'DATA 260 Homework 5 - Grocery Recall API'. The left sidebar lists the collection items, including 'POST 01 Login', 'GET 02 List recalls', 'GET 08 Fixed eager load - 10 records', 'GET 12 Fixed eager load - 200 records', 'POST 03 Create recall', 'GET 04 Read created recall', 'GET 11 Naive N+1 - 200 records', 'PUT 05 Update recall', 'GET 09 Naive N+1 - 50 records', 'GET 07 Naive N+1 - 10 records', 'GET 10 Fixed eager load - 50 records', 'DEL 06 Delete recall', 'DATA 260 Homework 5 - Grocery Recall API', 'Manufacturers', 'Recall notices', 'POST 01 Login', and 'DEL 12 Delete manufacturer'. The main panel displays the 'Run results' for the 'DATA 260 Homework 5 - Grocery Recall API' collection. The results show a successful run at 07:10:03 PM with 2 tests passed, 0 failed, 0 skipped, and 0 errors. The average response time is 22 ms. The tests include 'PUT ... > 10 Update recall' (200, 24 ms, 797 B), 'DELETE ... > 11 Delete recall' (204, 17 ms, 113 B), 'POST 01 Login' (200, 51 ms, 325 B), and 'DELETE 12 Delete manufacturer' (204, 22 ms, 113 B). The right sidebar shows a 'New Chat' section with a 'Run API smoke tests' button and a 'Describe what you need. Press @ for context, / for Skills.' prompt.

Source	Environment	Iterations	Duration	All tests	Errors
Runner	DATA 260 HW5 Local 3539	1	1s 731ms	2	0

Avg. Resp. Time: 22 ms

All 2 Passed 2 Failed 0 Skipped 0 Errors 0 Console log

PUT ... > 10 Update recall
http://127.0.0.1:8839/api/v1/recalls/6007 200 • 24 ms • 797 B

DELETE ... > 11 Delete recall
http://127.0.0.1:8839/api/v1/recalls/6007 204 • 17 ms • 113 B

POST 01 Login
http://127.0.0.1:8839/api/v1/auth/login 200 • 51 ms • 325 B

DELETE 12 Delete manufacturer
http://127.0.0.1:8839/api/v1/manufacturers/28 204 • 22 ms • 113 B

Successful recall update (200), recall delete (204), login (200), and manufacturer delete (204).

4. Live MySQL schema and row counts

The cumulative seed is stable after the UI and Postman demonstrations; temporary evidence records were removed.

```
Last login: Sun Oct 4 00:34:07 on ttys018
cd /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py
vedantvartak@Vedants-MacBook-Air ~ % cd /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539 && clear && .venv/bin/python scripts/capture_hw05_database_evidence.py

VEDANT VARTAK - DATA 260 HOMEWORK 5 - MYSQL EVIDENCE
timestamp_utc: 2026-10-04T07:34:11.556960+00:00
database: s3539_rel
tables: manufacturers, recall_events, recall_notices, sessions, users

row_counts:
  manufacturers: 12
  recall_notices: 500
  recall_events: 200
  users: 1
  sessions: 1

manufacturers_columns:
  id | INTEGER | nullable=False
  name | VARCHAR(160) | nullable=False
  contact_name | VARCHAR(160) | nullable=False
  contact_email | VARCHAR(255) | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

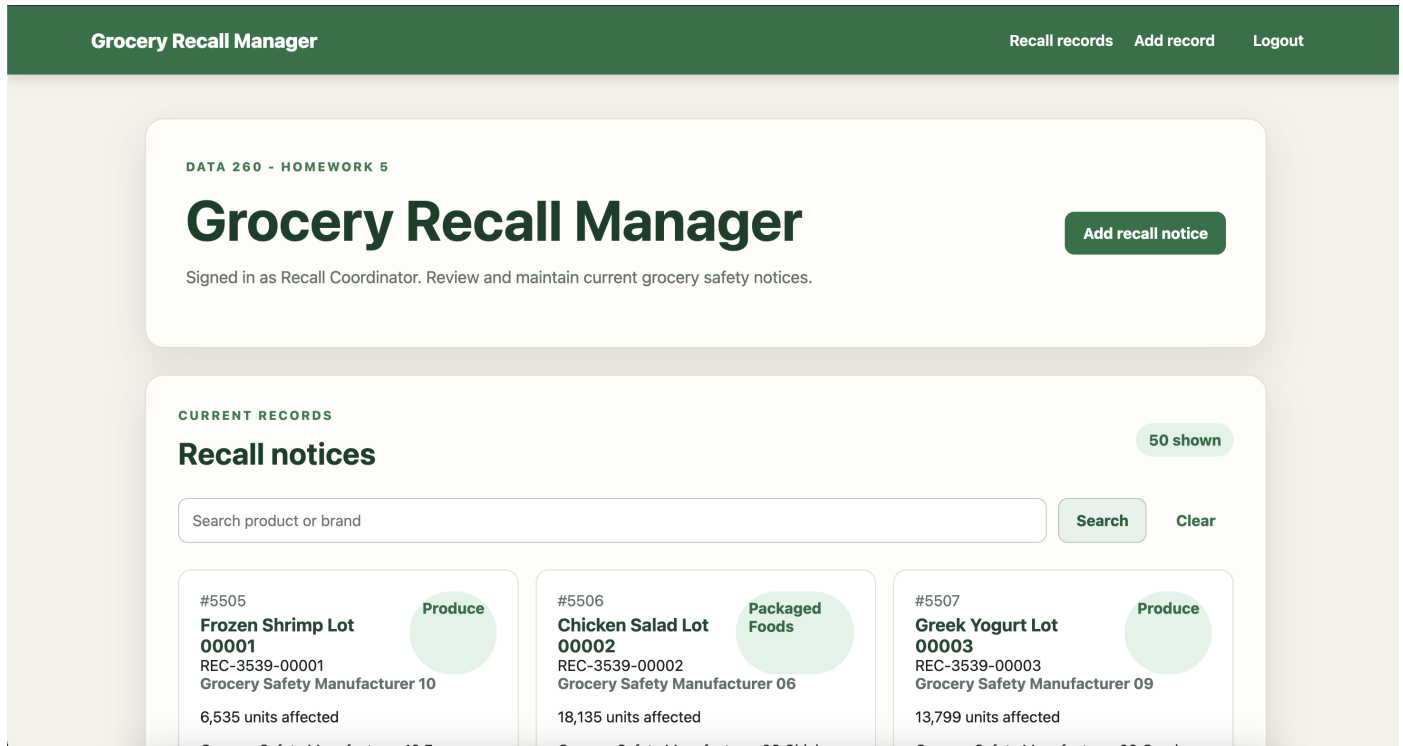
recall_notices_columns:
  id | INTEGER | nullable=False
  product_name | VARCHAR(160) | nullable=False
  recall_code | VARCHAR(32) | nullable=False
  units_affected | INTEGER | nullable=False
  manufacturer_id | INTEGER | nullable=False
  brand_name | VARCHAR(160) | nullable=False
  submitter_email | VARCHAR(255) | nullable=False
  category | VARCHAR(80) | nullable=False
  recall_details | TEXT | nullable=False
  terms_accepted | TINYINT | nullable=False
  created_at | DATETIME | nullable=False
  updated_at | DATETIME | nullable=False

recall_notices_foreign_keys:
  fk_recall_notices_manufacturers: ['manufacturer_id'] -> manufacturers['id'] onDelete=RESTRICT
Saved: /Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/database_evidence.txt
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Live schema evidence: 12 manufacturers, 500 recall notices, 200 related events, required columns, and RESTRICT foreign key.

5. Redux Home page

Home reads normalized application state through useSelector and dispatches the search thunk.



Authenticated Home page showing 50 database-backed recall cards and search controls.

Relevant Redux/React code in the same report frame

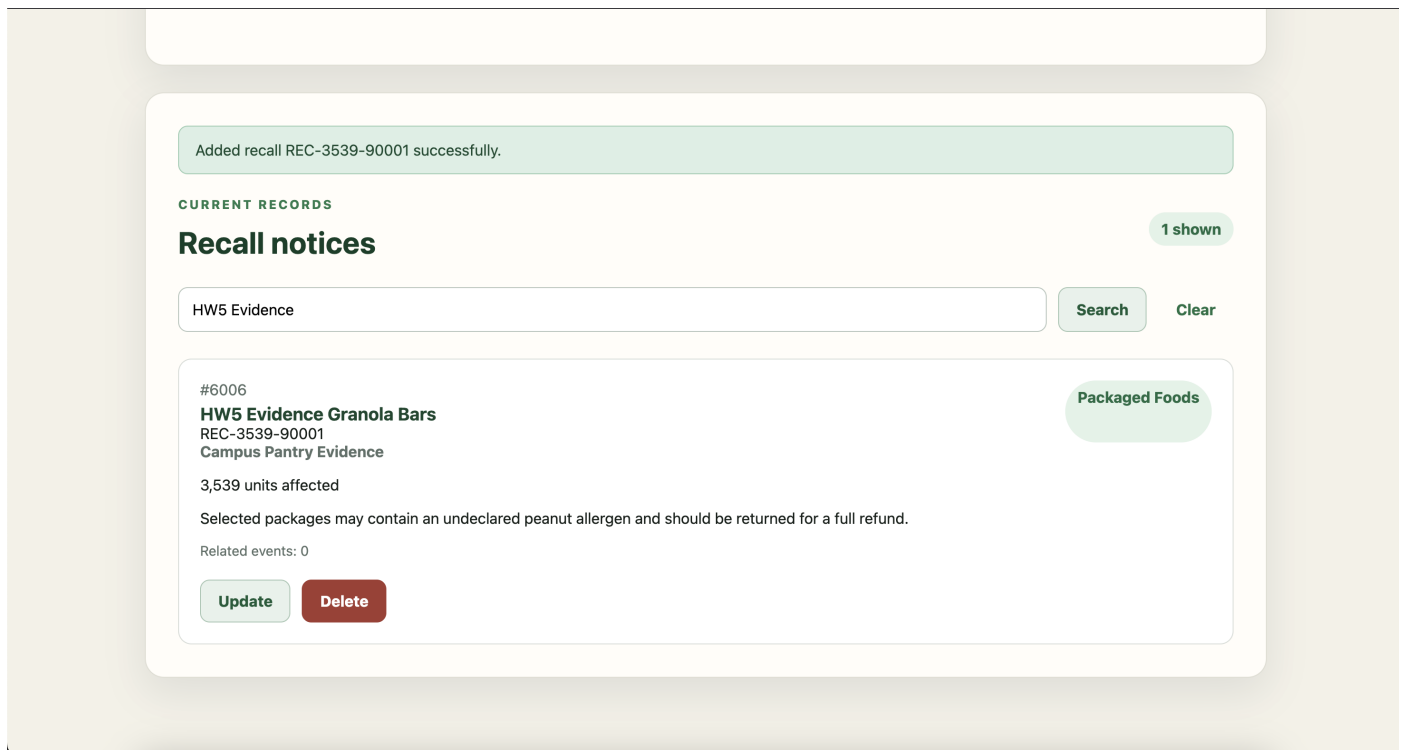
```
const dispatch = useDispatch();
const { items: records, loading, error, notice } =
  useSelector((state) => state.recalls);

function handleSearch(event) {
  event.preventDefault();
  dispatch(fetchRecalls(query));
}

<Link className="danger button-link"
  to={` /delete/${record.id}`}>Delete</Link>
```

6. Redux Create page

Create dispatches the asynchronous createRecall thunk and returns to Home only after unwrap confirms success.



Created REC-3539-90001; the Redux notice and persisted database record are visible.

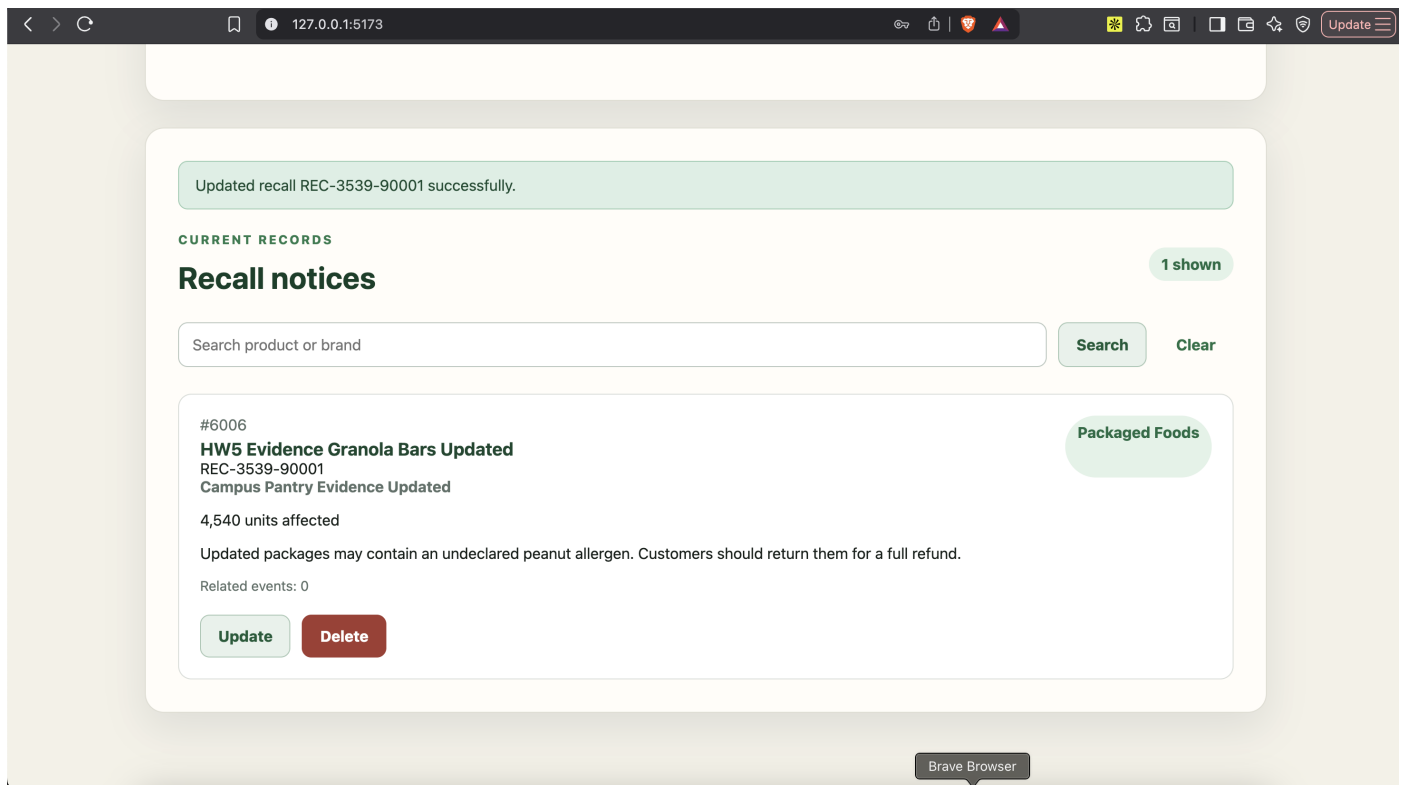
Relevant Redux/React code in the same report frame

```
async function handleCreate(payload) {
  await dispatch(createRecall(payload)).unwrap();
  navigate("/");
}

export const createRecall = createAsyncThunk(
  "recalls/create",
  async (payload, thunkApi) => {
    const response = await apiClient.post("/recalls", payload);
    return response.data;
  }
);
```

7. Redux Update page

Update loads one database record, dispatches updateRecall with its numeric ID, and replaces that item in Redux state.



The saved product, brand, units, and details are visibly updated after the API returns 200.

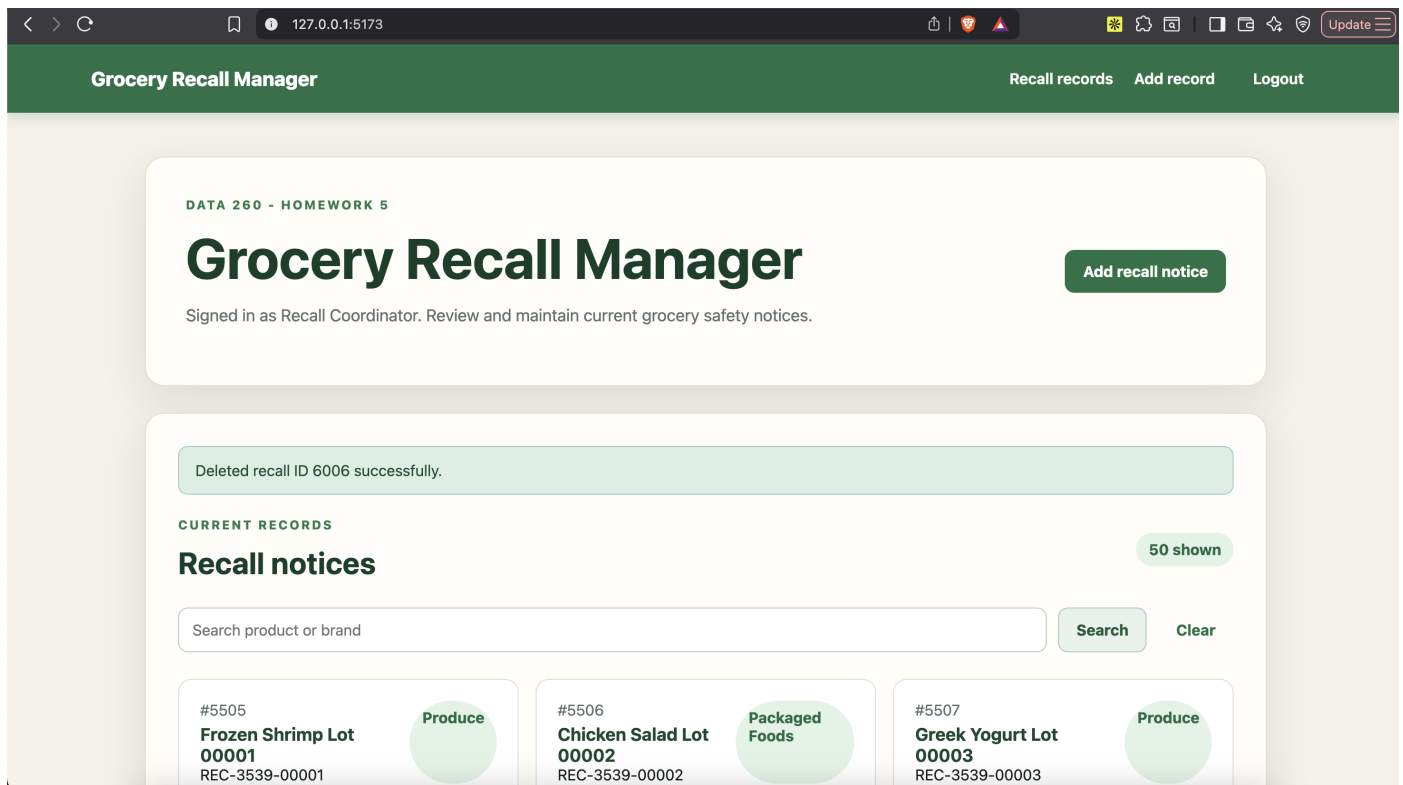
Relevant Redux/React code in the same report frame

```
async function handleUpdate(payload) {
  await dispatch(updateRecall({ id: Number(id), payload })).unwrap();
  navigate("/");
}

.addCase(updateRecall.fulfilled, (state, action) => {
  const index = state.items.findIndex(
    (item) => item.id === action.payload.id
  );
  if (index >= 0) state.items[index] = action.payload;
});
```

8. Redux Delete page

Delete uses a dedicated confirmation route and removes the record from Redux only after the API succeeds.



The success notice confirms temporary recall ID 6006 was deleted and the seeded 500-record state was restored.

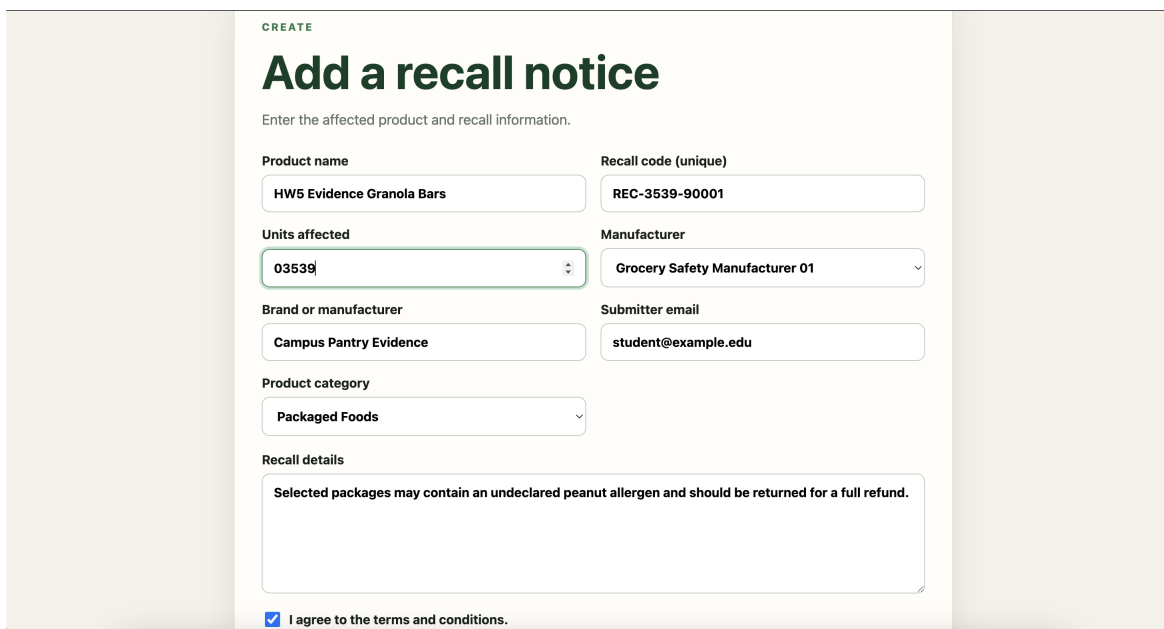
Relevant Redux/React code in the same report frame

```
async function handleDelete() {
  await dispatch(deleteRecall(Number(id))).unwrap();
  navigate("/");
}

.addCase(deleteRecall.fulfilled, (state, action) => {
  state.items = state.items.filter(
    (item) => item.id !== action.payload
  );
  state.notice = `Deleted recall ID ${action.payload} successfully.`;
});
```

9. Form and confirmation states

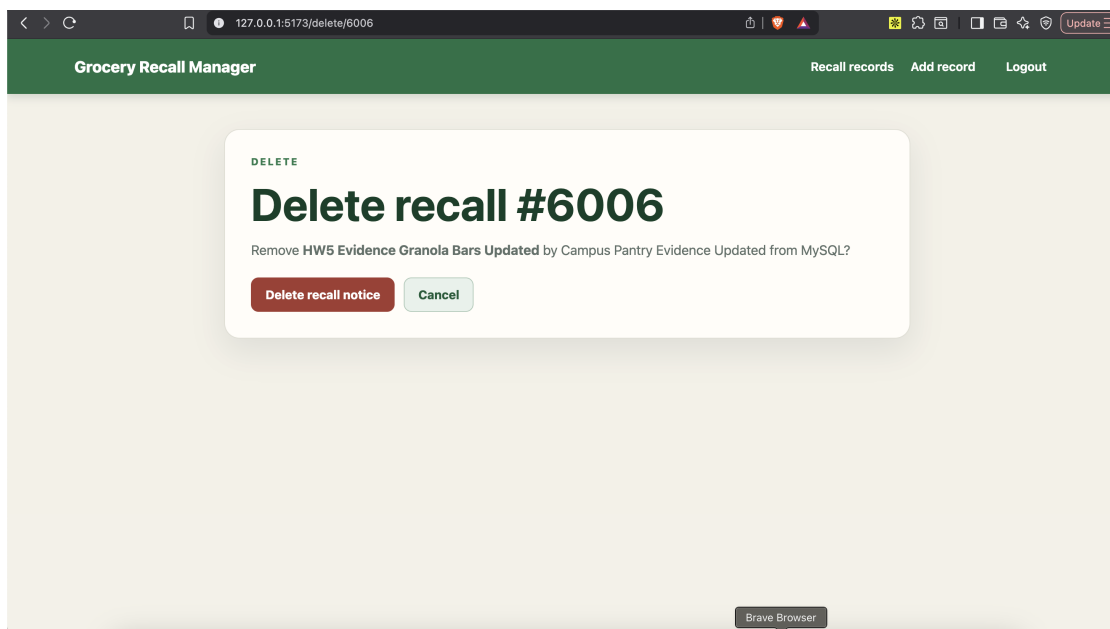
The input and confirmation screens provide additional evidence of complete user-facing CRUD behavior.



The screenshot shows a web form titled "Add a recall notice" with a "CREATE" label. The form is for entering product and recall information. It includes the following fields:

- Product name:** HW5 Evidence Granola Bars
- Recall code (unique):** REC-3539-90001
- Units affected:** 03539 (selected from a dropdown)
- Manufacturer:** Grocery Safety Manufacturer 01 (selected from a dropdown)
- Brand or manufacturer:** Campus Pantry Evidence
- Submitter email:** student@example.edu
- Product category:** Packaged Foods (selected from a dropdown)
- Recall details:** A text area containing "Selected packages may contain an undeclared peanut allergen and should be returned for a full refund."
- Agreement:** A checked checkbox labeled "I agree to the terms and conditions."

Create form populated with valid SID-specific evidence data and exact terms label.

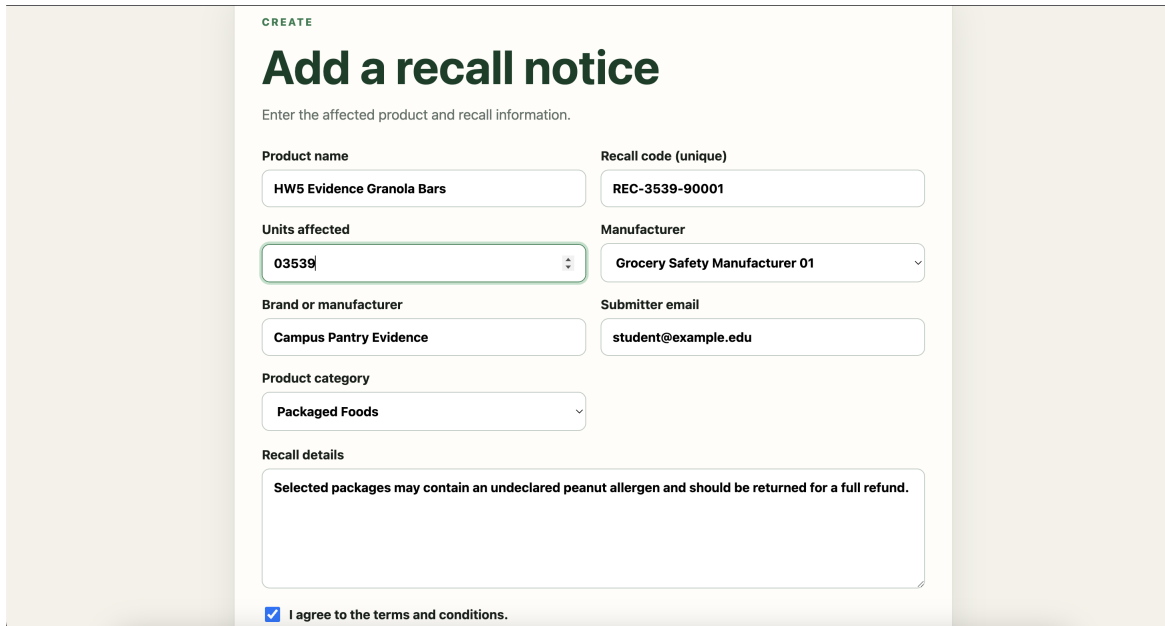


The screenshot shows a web browser window displaying a confirmation screen for deleting a recall. The browser's address bar shows the URL "127.0.0.1:5173/delete/6006". The page title is "Grocery Recall Manager". The confirmation screen has a "DELETE" label and the title "Delete recall #6006". The text below the title reads: "Remove HW5 Evidence Granola Bars Updated by Campus Pantry Evidence Updated from MySQL?". At the bottom of the confirmation screen are two buttons: "Delete recall notice" (in red) and "Cancel" (in light green). The browser's status bar at the bottom indicates "Brave Browser".

Dedicated confirmation screen prevents an accidental one-click delete.

10. Create and update form states

Both form paths reuse the validated RecordForm component while preserving distinct page routes.



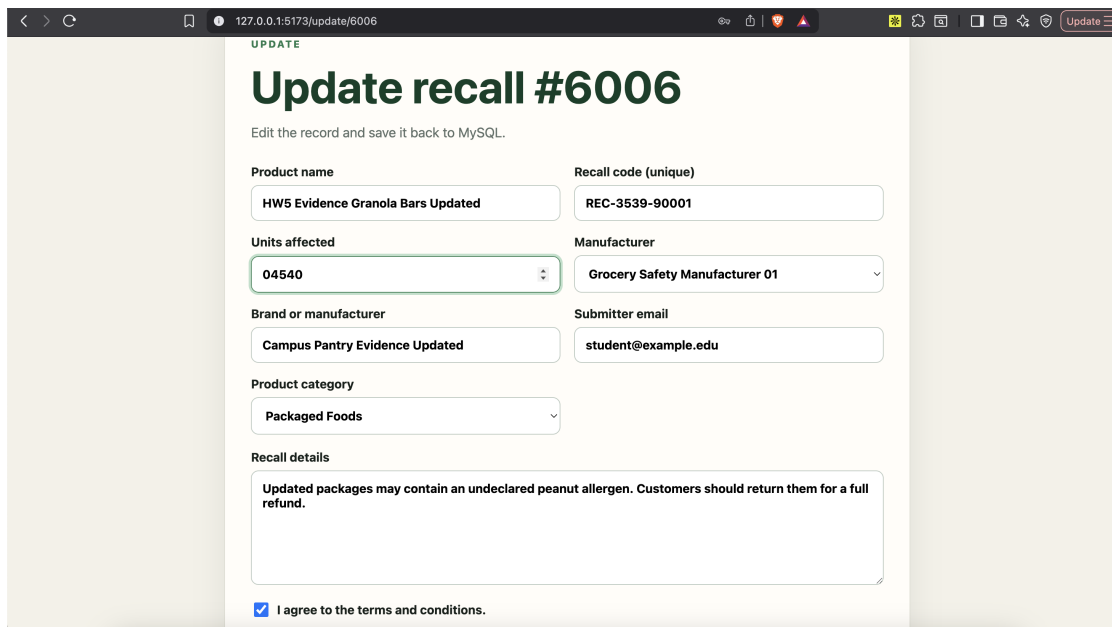
CREATE

Add a recall notice

Enter the affected product and recall information.

Product name	Recall code (unique)
HW5 Evidence Granola Bars	REC-3539-90001
Units affected	Manufacturer
03539	Grocery Safety Manufacturer 01
Brand or manufacturer	Submitter email
Campus Pantry Evidence	student@example.edu
Product category	
Packaged Foods	
Recall details	
Selected packages may contain an undeclared peanut allergen and should be returned for a full refund.	
☒ I agree to the terms and conditions.	

Create form before submission, including the exact terms-and-conditions control.



UPDATE

Update recall #6006

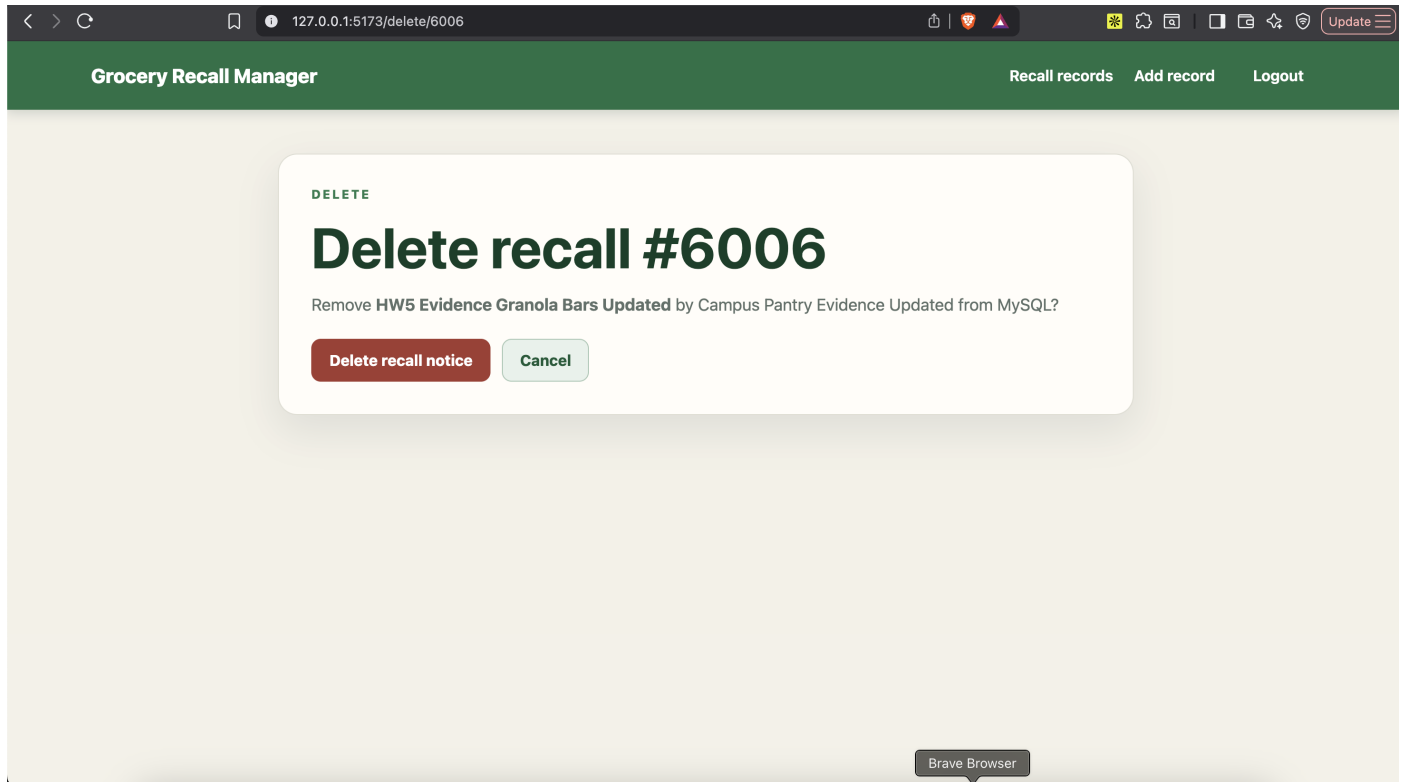
Edit the record and save it back to MySQL.

Product name	Recall code (unique)
HW5 Evidence Granola Bars Updated	REC-3539-90001
Units affected	Manufacturer
04540	Grocery Safety Manufacturer 01
Brand or manufacturer	Submitter email
Campus Pantry Evidence Updated	student@example.edu
Product category	
Packaged Foods	
Recall details	
Updated packages may contain an undeclared peanut allergen. Customers should return them for a full refund.	
☒ I agree to the terms and conditions.	

Update page loaded persisted values and accepted the revised data.

11. Explicit delete confirmation

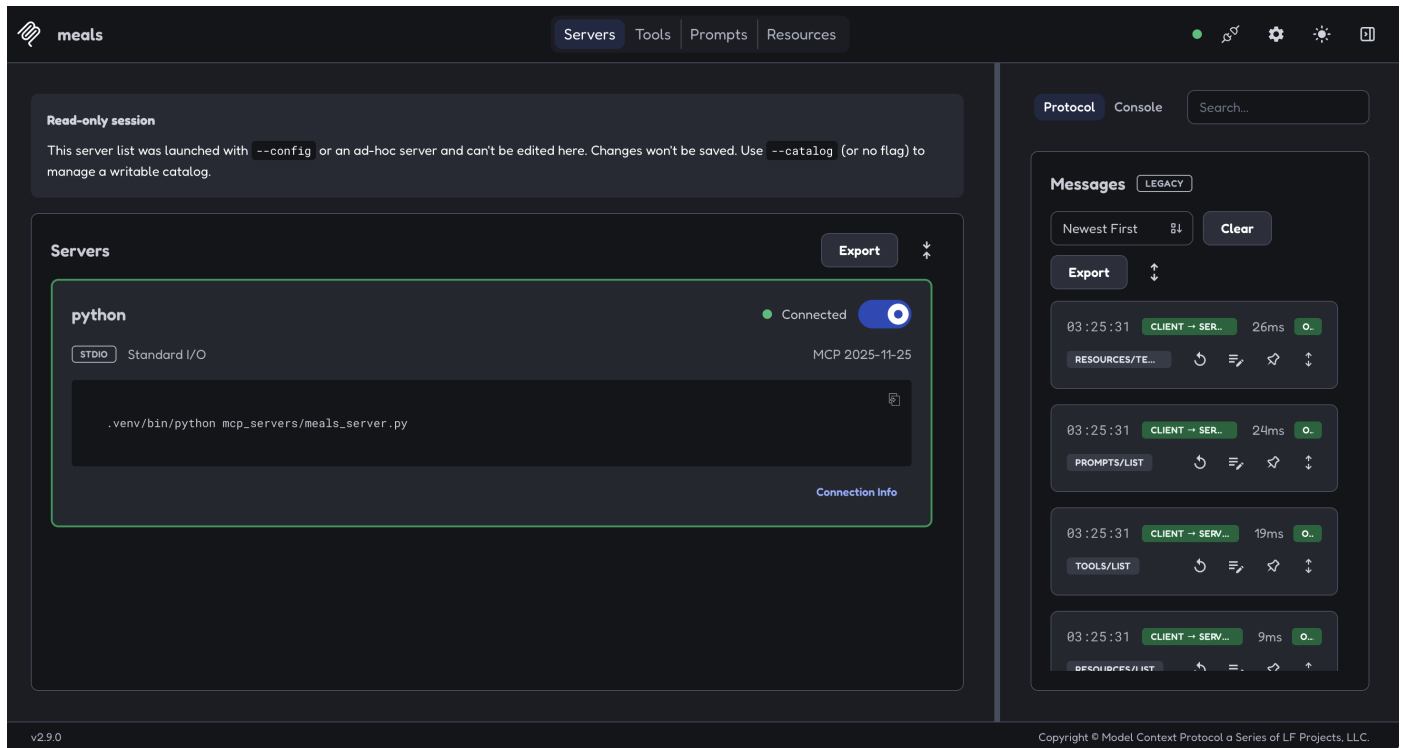
The destructive action is isolated behind a named confirmation page with a Cancel path.



Delete confirmation for temporary evidence record 6006 before the successful cleanup shown earlier.

12. MealDB MCP server connection

The Inspector connected over stdio using the project virtual environment and exposed exactly four required tools.



Connected MealDB MCP server in Inspector; tools, prompts, and resources inventories completed successfully.

13. MealDB tools - name and ingredient

Both discovery tools returned structured JSON from the public MealDB API.

The screenshot shows the MealDB interface with the 'Tools' tab selected. The 'search_meals_by_name' tool is active. The 'Results' panel shows a single JSON object for 'Spicy Arrabiata Penne'.

```
1 {
2   "meals": [
3     {
4       "id": "52771",
5       "name": "Spicy Arrabiata Penne",
6       "area": "Italian",
7       "category": "Vegetarian",
8       "thumb": "https://www.themealdb.com/images/media/meals/ustsqw14"
9     }
10  ],
11  "message": null
12 }
```

The 'Structured Output' panel shows the same JSON object.

search_meals_by_name returned Spicy Arrabiata Penne for the name query.

The screenshot shows the MealDB interface with the 'Tools' tab selected. The 'meals_by_ingredient' tool is active. The 'Results' panel shows a list of five chicken meals.

```
1 {
2   "meals": [
3     {
4       "id": "52940",
5       "name": "Brown Stew Chicken",
6       "thumb": "https://www.themealdb.com/images/media/meals/sypxpx15"
7     },
8     {
9       "id": "53161",
10      "name": "Chicken & chorizo rice pot",
11      "thumb": "https://www.themealdb.com/images/media/meals/fk80jp17"
12     },
13     {
14       "id": "52846",
15       "name": "Chicken & mushroom Hotpot",
16       "thumb": "https://www.themealdb.com/images/media/meals/uuspp15"
17     },
18     {
19       "id": "52796",
20       "name": "Chicken Alfredo Primavera",
21       "thumb": "https://www.themealdb.com/images/media/meals/syqypv14"
22     },
23     {
24       "id": "52934",
25       "name": "Chicken Basquaise",
26       "thumb": "https://www.themealdb.com/images/media/meals/wruvqv15"
27     }
28  ],
29  "message": null
30 }
```

The 'Structured Output' panel shows the same JSON object.

meals_by_ingredient returned five chicken meals with IDs, names, and thumbnails.

14. MealDB tools - details and random

The remaining tools return complete meal objects, including normalized ingredient arrays.

The screenshot shows the MealDB interface with the 'Tools' tab selected. The 'meal_details' tool is highlighted in the 'Tools' sidebar. The 'Results' panel displays a JSON object for meal ID 52771, 'Spicy Arrabiata Penne'. The JSON includes fields for id, name, category, area, instructions, image, source, youtube, and a normalized ingredients array. The ingredients array lists items like 'penne rigate', 'olive oil', 'garlic', 'chopped tomatoes', 'red chilli flakes', 'italian seasoning', and their respective measures. The 'Messages' panel on the right shows a log of client-server interactions.

```

1- {
2-   "id": "52771",
3-   "name": "Spicy Arrabiata Penne",
4-   "category": "Vegetarian",
5-   "area": "Italian",
6-   "instructions": "Bring a large pot of water to a boil. Add kosher s",
7-   "image": "https://www.themealdb.com/images/media/meals/ustsqw146825",
8-   "source": null,
9-   "youtube": "https://www.youtube.com/watch?v=1IszT-guI08",
10-  "ingredients": [
11-    {
12-      "name": "penne rigate",
13-      "measure": "1 pound"
14-    },
15-    {
16-      "name": "olive oil",
17-      "measure": "1/4 cup"
18-    },
19-    {
20-      "name": "garlic",
21-      "measure": "3 cloves"
22-    },
23-    {
24-      "name": "chopped tomatoes",
25-      "measure": "1 tin"
26-    },
27-    {
28-      "name": "red chilli flakes",
29-      "measure": "1/2 teaspoon"
30-    },
31-    {
32-      "name": "italian seasoning",
33-      "measure": "1/2 teaspoon"
34-    }
35-  ]
36- }

```

meal_details returned instructions, links, and measured ingredients for meal 52771.

The screenshot shows the MealDB interface with the 'Tools' tab selected. The 'random_meal' tool is highlighted in the 'Tools' sidebar. The 'Results' panel displays a JSON object for meal ID 52982, 'Spaghetti alla Carbonara'. The JSON includes fields for id, name, category, area, instructions, image, source, youtube, and a normalized ingredients array. The ingredients array lists items like 'Spaghetti', 'Egg Yolks', 'Salt', 'Bacon', 'Pecorino', and 'Black Pepper' with their respective measures. The 'Messages' panel on the right shows a log of client-server interactions.

```

1- {
2-   "id": "52982",
3-   "name": "Spaghetti alla Carbonara",
4-   "category": "Pasta",
5-   "area": "Italian",
6-   "instructions": "STEP 1\\nPut a large saucepan of water on to boil",
7-   "image": "https://www.themealdb.com/images/media/meals/1lcbn0157426",
8-   "source": "https://www.bbcgoodfood.com/recipes/ultimate-spaghetti-c",
9-   "youtube": "https://www.youtube.com/watch?v=w_T6jKRvhlk",
10-  "ingredients": [
11-    {
12-      "name": "Spaghetti",
13-      "measure": "320g"
14-    },
15-    {
16-      "name": "Egg Yolks",
17-      "measure": "6"
18-    },
19-    {
20-      "name": "Salt",
21-      "measure": "As required"
22-    },
23-    {
24-      "name": "Bacon",
25-      "measure": "150g"
26-    },
27-    {
28-      "name": "Pecorino",
29-      "measure": "50g"
30-    },
31-    {
32-      "name": "Black Pepper",
33-      "measure": "As required"
34-    }
35-  ]
36- }

```

random_meal returned Spaghetti alla Carbonara with complete structured details.

15. Domain tool contract - search

The domain MCP server exposes exactly search, detail, and aggregate, all using one stable JSON envelope.

The screenshot shows the MCP client interface for the 's3539_rel-recall-tools' server. The 'Tools' panel on the left has a search bar and three buttons: 'search', 'detail', and 'aggregate'. The 'search' button is selected. The 'Results' panel in the center displays a JSON response with an array of matches. The 'Messages' panel on the right shows a list of messages, including the search request and response.

```
1- {
2-   "ok": true,
3-   "data": {
4-     "matches": [
5-       {
6-         "id": 5505,
7-         "recall_code": "REC-3539-00001",
8-         "product_name": "Frozen Shrimp Lot 00001",
9-         "brand_name": "Grocery Safety Manufacturer 10",
10        "category": "Produce",
11        "units_affected": 6535
12      },
13      {
14        "id": 5508,
15        "recall_code": "REC-3539-00004",
16        "product_name": "Frozen Shrimp Lot 00004",
17        "brand_name": "Grocery Safety Manufacturer 09",
18        "category": "Dairy and Refrigerated",
19        "units_affected": 13996
20      },
21      {
22        "id": 5528,
23        "recall_code": "REC-3539-00024",
24        "product_name": "Frozen Shrimp Lot 00024",
25        "brand_name": "Grocery Safety Manufacturer 01",
26        "category": "Meat and Seafood",
27        "units_affected": 13039
28      },
29      {
30        "id": 5544,
31        "recall_code": "REC-3539-00040",
32        "product_name": "Frozen Shrimp Lot 00040",
33        "brand_name": "Grocery Safety Manufacturer 10",
34        "category": "Produce"
35      }
36    ]
37   }
38 }
```

Valid search for shrimp returned ok=true and SID-specific recall matches.

The screenshot shows the MCP client interface for the 's3539_rel-recall-tools' server. The 'Tools' panel on the left has a search bar and three buttons: 'search', 'detail', and 'aggregate'. The 'search' button is selected. The 'Results' panel in the center displays a JSON response with an error message. The 'Messages' panel on the right shows a list of messages, including the search request and response.

```
1- {
2-   "ok": false,
3-   "data": null,
4-   "error": "Invalid search input: query: String should have at least 2 characters"
5- }
```

One-character input returned ok=false, data=null, and a clean validation error.

16. Domain tool contract - detail

Detail resolves a positive integer recall ID and includes the related manufacturer object.

The screenshot displays the 's3539_rel-recall-tools' interface. On the left, the 'Tools' panel shows a search bar and a list of tools: 'Paginated', 'search', 'detail' (selected), and 'aggregate'. The 'Results' panel in the center shows a JSON response for the 'detail' tool call. The response includes fields such as 'id', 'recall_code', 'product_name', 'brand_name', 'category', 'units_affected', 'recall_details', and 'manufacturer'. The 'Structured Output' panel below it shows a simplified version of the same data. The 'Messages' panel on the right shows a log of client-to-server messages.

```
1- {
2  "ok": true,
3  "data": {
4    "id": 5505,
5    "recall_code": "REC-3539-00001",
6    "product_name": "Frozen Shrimp Lot 00001",
7    "brand_name": "Grocery Safety Manufacturer 10",
8    "category": "Produce",
9    "units_affected": 6535,
10   "recall_details": "Grocery Safety Manufacturer 10 Frozen Shrimp 1
11   "id": 24,
12   "name": "Grocery Safety Manufacturer 10",
13   "contact_name": "Safety Contact 10"
14 },
15 },
16 },
17 "error": null
18 }
```

```
1- {
2  "ok": true,
3  "data": {
4    "id": 5505,
5    "recall_code": "REC-3539-00001",
6    "product_name": "Frozen Shrimp Lot 00001",
7    "brand_name": "Grocery Safety Manufacturer 10",
8    "category": "Produce",
9    "units_affected": 6535,
10   "recall_details": "Grocery Safety Manufacturer 10 Frozen Sh
11   "manufacturer": {
```

Valid detail for recall 5505 returned recall and manufacturer fields.

The screenshot displays the 's3539_rel-recall-tools' interface. On the left, the 'Tools' panel shows a search bar and a list of tools: 'Paginated', 'search', 'detail' (selected), and 'aggregate'. The 'Results' panel in the center shows a JSON response for the 'detail' tool call. The response includes fields such as 'id', 'recall_code', 'product_name', 'brand_name', 'category', 'units_affected', 'recall_details', and 'manufacturer'. The 'Structured Output' panel below it shows a simplified version of the same data. The 'Messages' panel on the right shows a log of client-to-server messages.

```
1- {
2  "ok": false,
3  "data": null,
4  "error": "Invalid detail input: recall_id: Input should be greater t
5 }
```

```
1- {
2  "ok": false,
3  "data": null,
4  "error": "Invalid detail input: recall_id: Input should be gro
5 }
```

ID zero was rejected through the same error envelope without an exception.

17. Domain tool contract - aggregate

Aggregate restricts grouping to approved, non-sensitive dimensions.

The screenshot shows the 's3539_rel-recall-tools' interface. In the 'Tools' panel, 'aggregate' is selected. The 'Results' panel displays the following JSON response:

```

1 {
2   "ok": true,
3   "data": {
4     "group_by": "category",
5     "groups": [
6       {
7         "label": "Dairy and Refrigerated",
8         "recall_count": 127,
9         "total_units": 1198565
10      },
11      {
12        "label": "Meat and Seafood",
13        "recall_count": 130,
14        "total_units": 1327145
15      },
16      {
17        "label": "Packaged Foods",
18        "recall_count": 131,
19        "total_units": 1389038
20      },
21      {
22        "label": "Produce",
23        "recall_count": 112,
24        "total_units": 1098442
25      }
26    ]
27  },
28   "error": null
29 }

```

The 'Messages' panel on the right shows a log of the tool call, including the timestamp, client/server, and duration.

Category aggregation returned recall_count and total_units for all four categories.

The screenshot shows the 's3539_rel-recall-tools' interface. In the 'Tools' panel, 'aggregate' is selected. The 'Results' panel displays the following JSON response:

```

1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid aggregate input: group_by: String should match pat
5 }

```

The 'Structured Output' panel below the results shows the same error message. The 'Messages' panel on the right shows a log of the tool call, including the timestamp, client/server, and duration.

The unsupported submitter dimension was rejected with ok=false.

18. Retry policy and fault-injection results

The experiment used VERIFY_SEED 263539 and exactly 150 calls: 50 each at 0%, 20%, and 50% injected failure.

Injected failure	Calls	Success	Mean latency	p99 latency
0%	50	100%	0.000 ms	0.001 ms
20%	50	98%	0.276 ms	3.715 ms
50%	50	92%	0.882 ms	3.791 ms

Policy: at most three attempts, 100 ms total timeout, and bounded exponential delays of 1 ms then 2 ms (4 ms cap). Normal calls have negligible delay. At 50% injected failure, 92% still completed, while the remaining failures returned a clean error after exhausting the fixed attempt budget. A batch workload could tolerate more attempts and jitter; the interactive assistant favors a short, predictable response ceiling.

```
< .venv/bin/python scripts/run_hw05_fault_experiment.py
Vedant Vartak - DATA 260 Homework 5 fault-injection experiment
VERIFY_SEED=263539; calls=150
{"calls": 50, "injected_failure_rate": 0.0, "mean_latency_ms": 0.0, "p99_latency_ms": 0.001, "success_rate": 1.0}
{"calls": 50, "injected_failure_rate": 0.2, "mean_latency_ms": 0.276, "p99_latency_ms": 3.715, "success_rate": 0.98}
{"calls": 50, "injected_failure_rate": 0.5, "mean_latency_ms": 0.882, "p99_latency_ms": 3.791, "success_rate": 0.92}
raw_csv=/Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/fault_injection_calls.csv
<"EXHAUSTED RETRIES:",next(x for x in r if x["success"]=="False"))'
Vedant Vartak - Retry behavior evidence
IMMEDIATE SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.0', 'call_index': '1', 'success': 'True', 'attempts': '1', 'attempt_outcomes': 'success', 'latency_ms': '0.002', 'error': ''}
RETRY SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '13', 'success': 'True', 'attempts': '2', 'attempt_outcomes': 'failure|success', 'latency_ms': '1.266', 'error': ''}
EXHAUSTED RETRIES: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '46', 'success': 'False', 'attempts': '3', 'attempt_outcomes': 'failure|failure|failure', 'latency_ms': '3.773', 'error': 'operation failed after 3 attempts: injected transient storage failure'}
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Representative rows show immediate success, failure-then-success, and three exhausted attempts; the summary above is derived from all 150 CSV rows.

19. Offline tool and safety tests

The offline runner is deterministic and requires no external network or language model.

```
< .venv/bin/python scripts/run_hw05_fault_experiment.py
Vedant Vartak - DATA 260 Homework 5 fault-injection experiment
VERIFY_SEED=263539; calls=150
{"calls": 50, "injected_failure_rate": 0.0, "mean_latency_ms": 0.0, "p99_latency_ms": 0.001, "success_rate": 1.0}
{"calls": 50, "injected_failure_rate": 0.2, "mean_latency_ms": 0.276, "p99_latency_ms": 3.715, "success_rate": 0.98}
{"calls": 50, "injected_failure_rate": 0.5, "mean_latency_ms": 0.882, "p99_latency_ms": 3.791, "success_rate": 0.92}
raw_csv=/Users/vedantvartak/Documents/Codex/2026-08-25/c/outputs/data260-3539/reports/hw05/raw/fault_injection_calls.csv
<"EXHAUSTED RETRIES:",next(x for x in r if x["success"]=="False"))'
Vedant Vartak - Retry behavior evidence
IMMEDIATE SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.0', 'call_index': '1', 'success': 'True', 'attempts': '1', 'attempt_outcomes': 'success', 'latency_ms': '0.002', 'error': ''}
RETRY SUCCESS: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '13', 'success': 'True', 'attempts': '2', 'attempt_outcomes': 'failure|success', 'latency_ms': '1.266', 'error': ''}
EXHAUSTED RETRIES: {'verify_seed': '263539', 'injected_failure_rate': '0.2', 'call_index': '46', 'success': 'False', 'attempts': '3', 'attempt_outcomes': 'failure|failure|failure', 'latency_ms': '3.773', 'error': 'operation failed after 3 attempts: injected transient storage failure'}
< clear; .venv/bin/python scripts/test_hw05_tools.py
Vedant Vartak - DATA 260 Homework 5 offline tests
PASS test_search_valid
PASS test_search_invalid
PASS test_detail_valid
PASS test_detail_invalid
PASS test_aggregate_valid
PASS test_aggregate_invalid
PASS test_safety_rule_allowed
PASS test_safety_rule_blocked
PASS test_agent_max_steps
SUMMARY 9/9 tests passed
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Nine of nine tests passed: valid/invalid calls for all tools, allowed/blocked safety cases, and max-step termination.

20. Local Ollama agent scenarios

All scenarios used qwen3:8b, temperature 0, model seed 3539, and MAX_STEPS=6.

```
Vedant Vartak - DATA 260 Homework 5 agent scenarios
MODEL=qwen3:8b MAX_STEPS=6 SCENARIOS=4
Scenario 1 | steps 2 | tool_calls 1 | normal_completion
  Three shrimp recalls were found: 1. **Grocery Safety Manufacturer 10** (Produce) - Frozen Shrimp Lot 00001 (REC-3539-00001), 6,535 units affected. 2. **Grocery Safety Manufacturer 09** (Dairy and Refrigerated) - Frozen Shrimp Lot 00004 (REC-3539-00004), 13,996 units affected. 3.
Scenario 2 | steps 2 | tool_calls 1 | normal_completion
  {'product': 'Frozen Shrimp Lot 00001', 'manufacturer': 'Grocery Safety Manufacturer 10 (Safety Contact 10)'}
Scenario 3 | steps 2 | tool_calls 1 | normal_completion
  Recalls grouped by category (min_units=0): Packaged Foods had the highest total units (1,389,038) and 131 recalls. Meat and Seafood followed with 130 recalls (1,327,145 units). Dairy and Refrigerated (127 recalls, 1,198,565 units) and Produce (112 recalls, 1,098,442 units) complete
Scenario 4 | steps 1 | tool_calls 1 | safety_rule_block
  Safety rule blocked search: limit cannot exceed 10 records
PASS: 3 normal completions + 1 safety-rule block
vedantvartak@Vedants-MacBook-Air data260-3539 %
```

Three scenarios completed normally after one grounded tool call; the fourth stopped immediately at the safety boundary.

21. Agent reflection

Selected run: Scenario 4 - deterministic safety-rule block.

I selected the fourth Ollama run because it demonstrates that the harness, not the language model, owns the safety boundary. The user prompt explicitly asked the agent to retrieve recall records with a limit of 25 and told it not to reduce that limit. In step 1, local qwen3:8b produced a structured action for the search tool with query set to an empty string and limit set to 25. The harness did not send that action directly to the database. Instead, `run_agent` routed it through the single `execute_tool(name, inputs)` entry point used by every agent tool call.

Inside `execute_tool`, the domain-specific safety rule checked the requested search limit before normal schema validation or repository access. Because the limit exceeded the allowed agent maximum of 10, the function returned the shared JSON envelope with `ok=false`, `data=null`, and the error 'Safety rule blocked search: limit cannot exceed 10 records.' It raised no exception and issued no SQL query.

The loop parsed this clean result, logged the tool name, input, complete result, step number, and run identifier to `agent_runs.jsonl`, and recognized the safety-error prefix. It then stopped immediately with `stop_reason` equal to `safety_rule_block`. The run therefore used one step and one tool-call attempt, never approached the six-step ceiling, and never gave the model a chance to bypass the policy. This run shows why centralized enforcement is stronger than asking the model to follow a prompt: even when the model follows the user's unsafe request exactly, deterministic application code blocks it consistently.

Word count: 264

22. Tool contracts under stress

Tool	Accepted input	Rejected input	Clean behavior
search	query 2-80 chars; limit 1-25	query='s'	Minimum-length error
detail	positive recall_id	recall_id=0	Positive-ID error
aggregate	category manufacturer	group_by='submitter'	Approved-dimension error

All three return {ok, data, error}. Pydantic validation is caught at the tool boundary so bad arguments never become uncaught exceptions. The Part 5 agent rule is intentionally narrower than the MCP schema: `execute_tool` blocks search limits above 10 to prevent bulk extraction, while direct interactive MCP use still permits up to 25.

AI-use disclosure

1. I used an AI assistant to translate the assignment checklist into a cumulative implementation plan, draft FastAPI/Redux/MCP code, and create repeatable validation scripts. I independently reviewed the requirements, ran the tests, inspected the artifacts, and captured the final evidence.
2. I independently verified that the supplied demo's relationship route order was unsuitable to copy directly: a dynamic `/ {course_id}` route appeared before `/by-instructor/{instructor_id}`, which can cause the static path to be parsed as an integer ID.
3. I detected the issue by reading the complete demo source after extracting the assignment requirements, then comparing FastAPI declaration-order matching with the required relationship endpoint. I also used production builds and offline tests instead of assuming generated code was correct.
4. I declared the static `/by-manufacturer/{manufacturer_id}` relationship route before the dynamic `/ {recall_id}` route. The cumulative suite and live API flow verify that relationship lookup coexists with individual recall lookup.

23. Submission file alignment

Required item	Repository location
Final report	reports/hw05/report.pdf
Run log	reports/hw05/RUN_LOG.txt
Metrics	reports/hw05/METRICS.md
AI disclosure	reports/hw05/AI_USE.md
Verification	reports/hw05/verification.json
Raw evidence	reports/hw05/raw/
Screenshots	reports/hw05/evidence/
Postman files	reports/hw05/postman/
Canvas upload copy	output/pdf/Vartak_HW5.pdf

The repository deliberately excludes temporary demonstration rows: UI record 6006 and Postman recall 6007/manufacture 28 were deleted after proof was captured. The final live database therefore matches the deterministic cumulative seed.

End of report